

1. Check Whether a Number is Even or Odd

Problem Statement

Write a JavaScript program that checks whether a given number is even or odd. A number is even if it is completely divisible by 2; otherwise, it is odd.

Source Code

```
// Program to check whether a number is even or odd

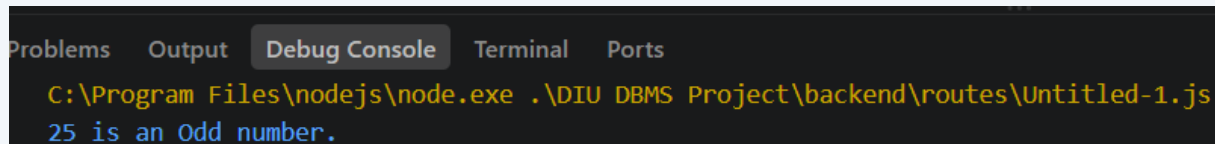
let number = 25;

if (number % 2 === 0) {
    console.log(number + " is an Even number.");
} else {
    console.log(number + " is an Odd number.");
}
```

Expected Output

25 is an Odd number.

Output Screenshot



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports
C:\Program Files\nodejs\node.exe .\DIU DBMS Project\backend\routes\Untitled-1.js
25 is an Odd number.
```

Explanation

The program stores 25 in the variable `number`. The modulus operator (%) returns the remainder after division by 2. If the remainder is 0, the number is even; otherwise, it is odd. Since $25 \% 2$ is 1, the program correctly identifies 25 as an odd number.

Concepts Demonstrated: Variables • Modulus operator • if/else • Decision-making

2. Find the Largest of Three Numbers

Problem Statement

Write a JavaScript program that takes three numbers and determines which number is the largest.

Source Code

```
// Program to find the largest of three numbers

let num1 = 45;
let num2 = 78;
let num3 = 32;

let largest;

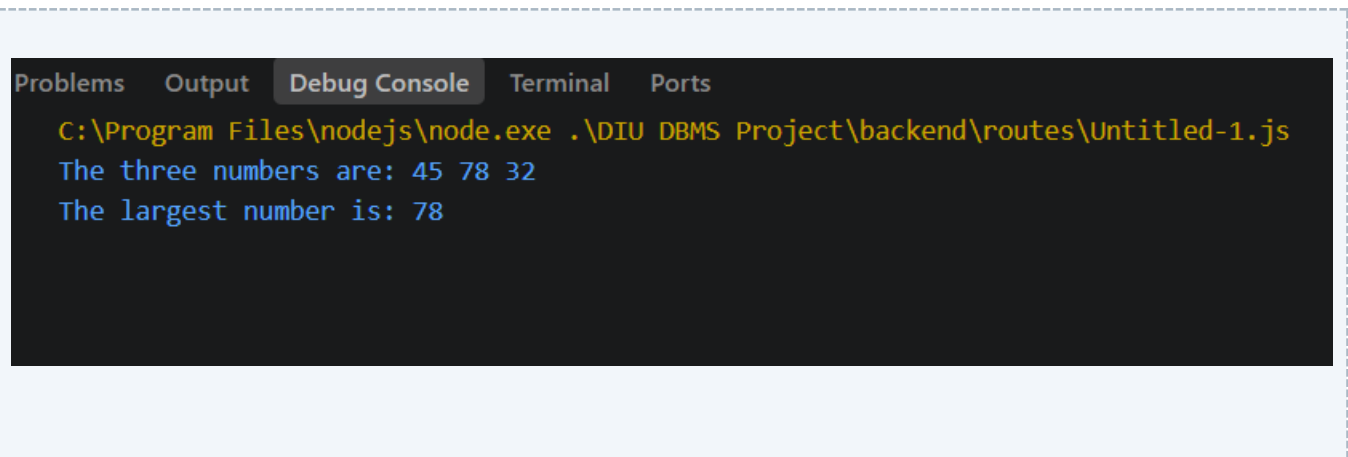
if (num1 >= num2 && num1 >= num3) {
    largest = num1;
} else if (num2 >= num1 && num2 >= num3) {
    largest = num2;
} else {
    largest = num3;
}

console.log("The three numbers are:", num1, num2, num3);
console.log("The largest number is:", largest);
```

Expected Output

```
The three numbers are: 45 78 32
The largest number is: 78
```

Output Screenshot



Explanation

The three values are stored in num1, num2, and num3. The if/else-if/else structure compares the values. The logical AND operator (&&) ensures that the selected number is greater than or equal to both other values. Therefore, 78 is identified as the largest.

Concepts Demonstrated: Comparison operators • Logical AND • Conditional branching

3. Reverse a String

Problem Statement

Write a JavaScript program that reverses a given string and displays both the original and reversed strings.

Source Code

```
// Program to reverse a string

let text = "JavaScript";

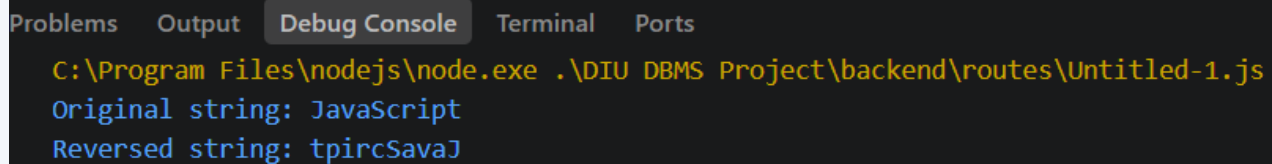
let reversedText = text.split("").reverse().join("");

console.log("Original string:", text);
console.log("Reversed string:", reversedText);
```

Expected Output

Original string: JavaScript
Reversed string: tpircSavaJ

Output Screenshot



The screenshot shows the VS Code interface with the 'Debug Console' tab selected. The output displays the file path 'C:\Program Files\nodejs\node.exe .\DIU DBMS Project\backend\routes\Untitled-1.js' followed by the program's output: 'Original string: JavaScript' and 'Reversed string: tpircSavaJ'.

Explanation

The `split("")` method converts the string into an array of characters. The `reverse()` method reverses the character order, and `join("")` combines the characters back into one string. Therefore, the reversed form of JavaScript is tpircSavaJ.

Concepts Demonstrated: String manipulation • `split()` • `reverse()` • `join()`

4. Count Vowels in a String

Problem Statement

Write a JavaScript program that counts the number of vowels present in a string. The vowels considered are a, e, i, o, and u.

Source Code

```
// Program to count vowels in a string

let text = "Hello JavaScript";
let count = 0;

for (let i = 0; i < text.length; i++) {
    let character = text[i].toLowerCase();

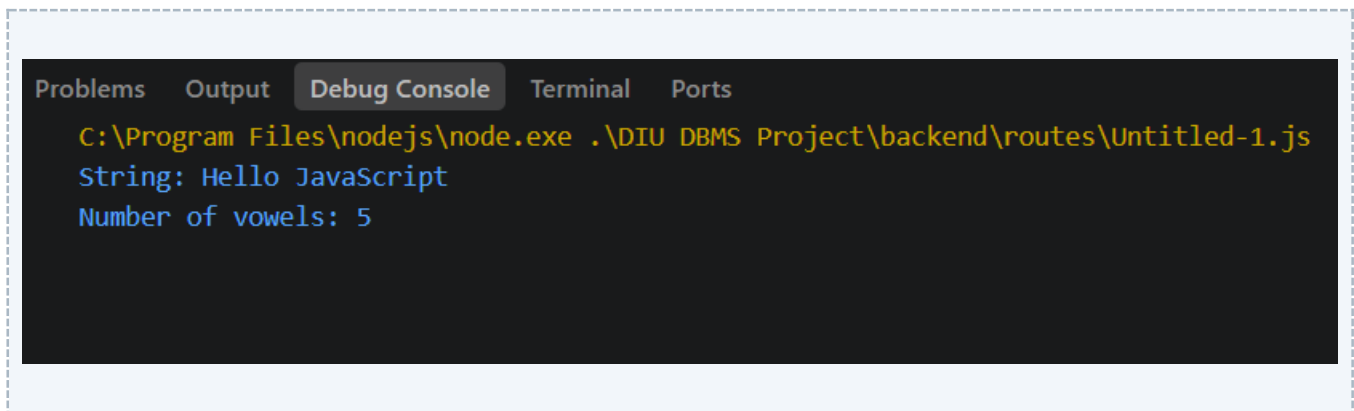
    if (
        character === "a" ||
        character === "e" ||
        character === "i" ||
        character === "o" ||
        character === "u"
    ) {
        count++;
    }
}

console.log("String:", text);
console.log("Number of vowels:", count);
```

Expected Output

```
String: Hello JavaScript
Number of vowels: 5
```

Output Screenshot



Explanation

A for loop examines every character in the string. Each character is converted to lowercase so uppercase and lowercase vowels can be checked consistently. When a character matches a, e, i, o, or u, the counter increases by one. The final count is 4.

Concepts Demonstrated: for loop • String indexing • toLowerCase() • Counter

5. Remove Duplicate Values from an Array

Problem Statement

Write a JavaScript program that removes duplicate values from an array and displays an array containing only unique values.

Source Code

```
// Program to remove duplicate values from an array

let numbers = [10, 20, 10, 30, 20, 40, 30, 50];

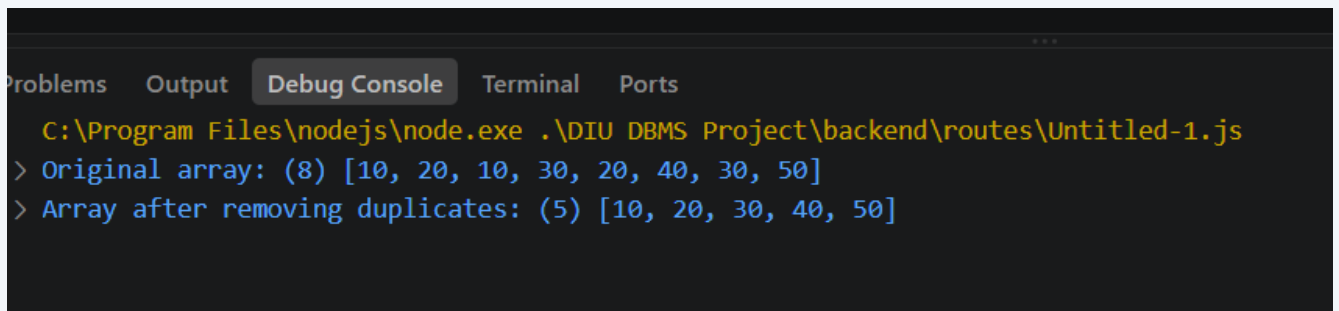
let uniqueNumbers = [...new Set(numbers)];

console.log("Original array:", numbers);
console.log("Array after removing duplicates:", uniqueNumbers);
```

Expected Output

```
Original array: [10, 20, 10, 30, 20, 40, 30, 50]
Array after removing duplicates: [10, 20, 30, 40, 50]
```

Output Screenshot



The screenshot shows a Node.js application window with the 'Debug Console' tab selected. The command prompt shows the execution of a JavaScript file. The output displays the original array with 8 elements, including duplicates, and the resulting array after removing duplicates, which contains 5 unique elements.

```
C:\Program Files\nodejs\node.exe .\DIU DBMS Project\backend\routes\Untitled-1.js
> Original array: (8) [10, 20, 10, 30, 20, 40, 30, 50]
> Array after removing duplicates: (5) [10, 20, 30, 40, 50]
```

Explanation

The Set object stores unique values, so duplicate elements are automatically removed. The spread operator (...) converts the Set back into an array. The resulting array contains only the unique values: 10, 20, 30, 40, and 50.

Q2. Introduction to Node.js

1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment that allows JavaScript code to run outside a web browser. It extends JavaScript beyond frontend web-page interaction and makes it possible to execute JavaScript on a computer or server.

Node.js is built on Google's V8 JavaScript engine. It is commonly used for backend applications, web servers, REST APIs, command-line tools, real-time applications, automation, and software testing.

Example command: Save JavaScript code in a file such as app.js and run it from the VS Code terminal using:
node app.js

2. How Does Node.js Work?

Node.js uses the V8 engine to execute JavaScript and follows an event-driven, non-blocking I/O model. When an asynchronous task is started, Node.js can continue processing other work rather than blocking the entire application.

```
console.log("Start");

setTimeout(() => {
  console.log("Task completed");
}, 2000);

console.log("End");
```

Start
End
Task completed

In this example, `setTimeout()` schedules the callback for later. Node.js continues executing the next statement, so “End” is printed before “Task completed”. The event loop coordinates the asynchronous callback when the timer is ready.

Core Components

Component	Purpose
V8 Engine	Executes JavaScript code.
Event Loop	Coordinates asynchronous callbacks and tasks.
Non-blocking I/O	Allows other work to continue while I/O is being handled.
Event-driven model	Lets completed operations trigger appropriate handlers.

3. Advantages of Node.js for QA Automation

JavaScript-based automation: QA engineers can write automated tests using JavaScript, making it convenient when applications are also developed with JavaScript.

Fast execution: The V8 engine provides high-performance JavaScript execution, supporting efficient test execution.

Rich testing ecosystem: Node.js supports modern tools and frameworks such as Playwright, Cypress, Jest, Mocha, WebdriverIO, and Supertest.

API testing: Node.js-based tools can send HTTP requests, validate response data, check status codes, and automate API test scenarios.

Browser automation: Suitable Node.js frameworks can automate navigation, clicking, form filling, validation, and screenshot capture.

CI/CD integration: Automated tests can run in continuous integration and delivery pipelines to help detect defects after code changes.

Large package ecosystem: npm provides a large collection of reusable packages for testing, reporting, automation, and development.

QA Automation Value: Node.js combines JavaScript support, asynchronous execution, automation libraries, API testing capabilities, and CI/CD integration, making it a practical platform for modern software testing workflows.

4. Difference Between Browser JavaScript and Node.js

Feature	Browser JavaScript	Node.js
Execution environment	Web browser	Node.js runtime on a computer/server
Primary purpose	Frontend and web-page interaction	Backend, APIs, automation, testing, and tools
DOM access	Yes	No direct browser DOM
window / document	Available in browser context	Not available by default
File-system access	Restricted for security	Available through Node.js APIs
Server-side use	Not the normal role	Yes
Database access	Usually through a backend/API	Can connect using appropriate drivers/packages
npm packages	Not the normal execution model	Directly supported
Common examples	DOM manipulation, UI behavior	REST APIs, servers, automation, test tools

Practical Examples

Browser JavaScript

```
document.getElementById("message").innerText = "Hello";
```

This works in a browser because the browser provides the DOM and the document object.

Node.js

```
const fs = require("fs");

fs.writeFileSync("example.txt", "Hello from Node.js!");

console.log("File created successfully.");
```

Node.js provides file-system APIs that allow a program to work with files directly.

Key Distinction: Browser JavaScript primarily interacts with web pages and browser APIs, while Node.js executes JavaScript outside the browser and provides runtime APIs for server-side, file-system, networking, automation, and testing tasks.